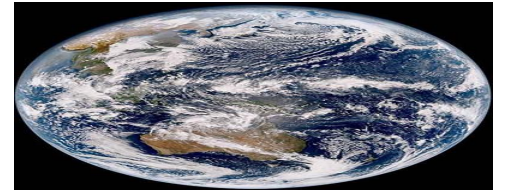
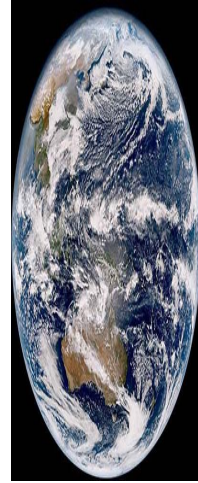


メディアコンピューティング2026

第7回 幾何学的変換

8-1 線形変換



8-1 線形変換

線形変換とは

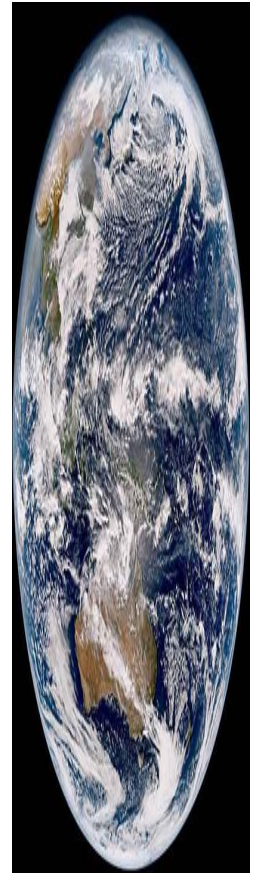
線形変換とは行列によって空間を変形させる事



×

$$\begin{bmatrix} 0.5 & 0 \\ 0 & 2 \end{bmatrix}$$

=



8-1 線形変換 行列

まず，線形変換の理解には行列が必要

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

例えば横に長く拡大する時は

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



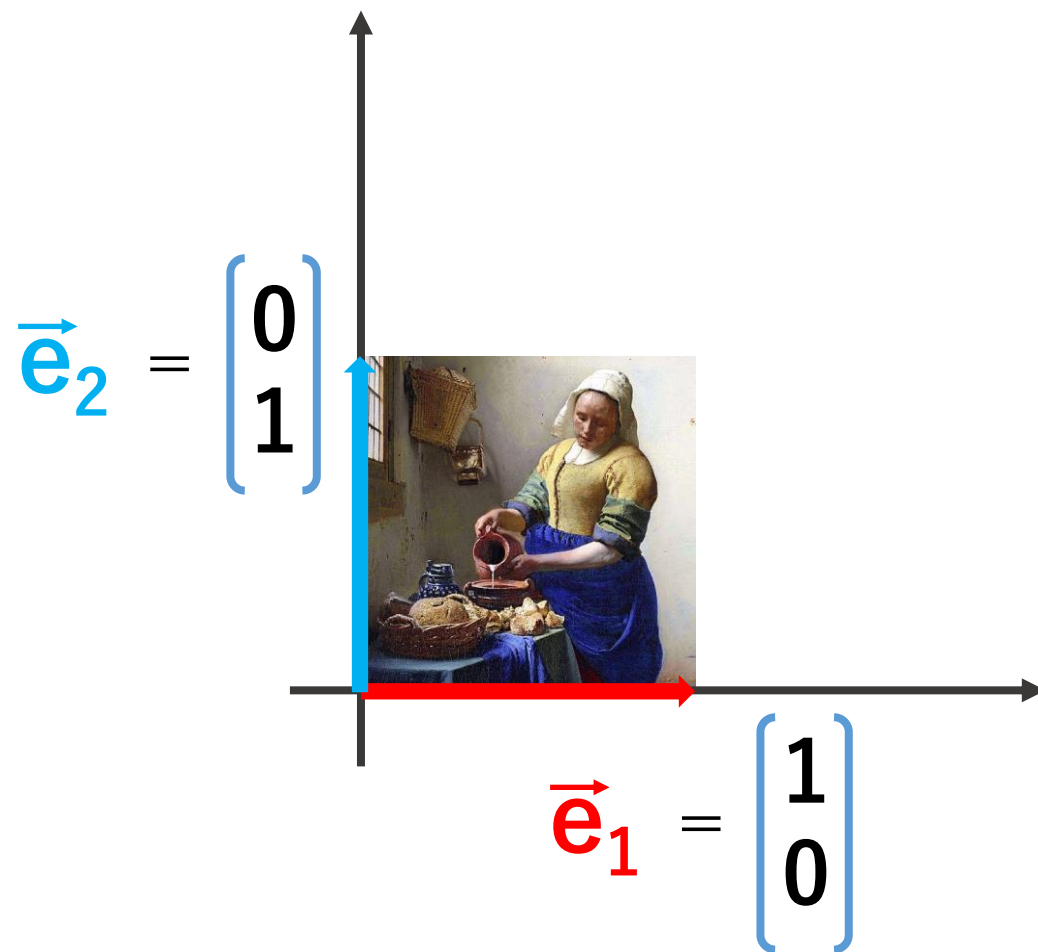
8-1 線形変換 ベクトル

さらに、行列の理解にベクトルが必要

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} f(\vec{e}_1) & f(\vec{e}_2) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



8-1 線形変換 ベクトル

実際に画像を変化させる

横の長さを半分にする

$$f(\vec{e}_1) = 0.5$$

$$f() = S_x = \begin{bmatrix} 0.5 & \\ & \end{bmatrix}$$

縦の長さを2倍にする

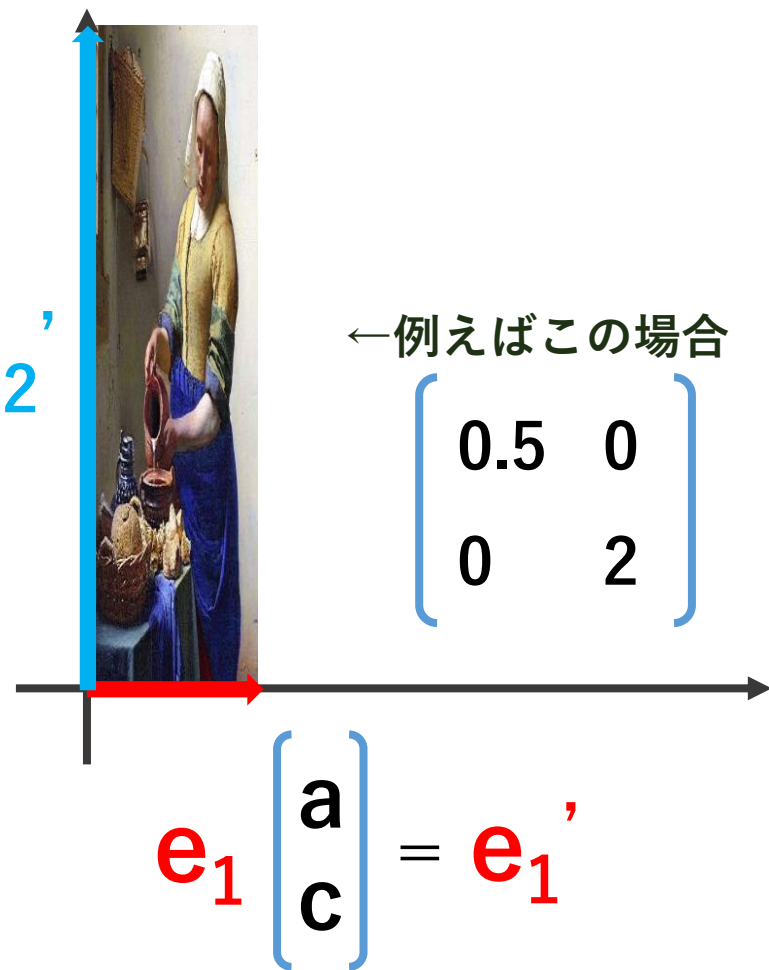
$$f(\vec{e}_2) = 2$$

$$f() = S_y = \begin{bmatrix} & \\ 0 & 2 \end{bmatrix}$$

$$\mathbf{e}_2 \begin{bmatrix} \mathbf{b} \\ \mathbf{d} \end{bmatrix} = \mathbf{e}_2'$$

←例えばこの場合

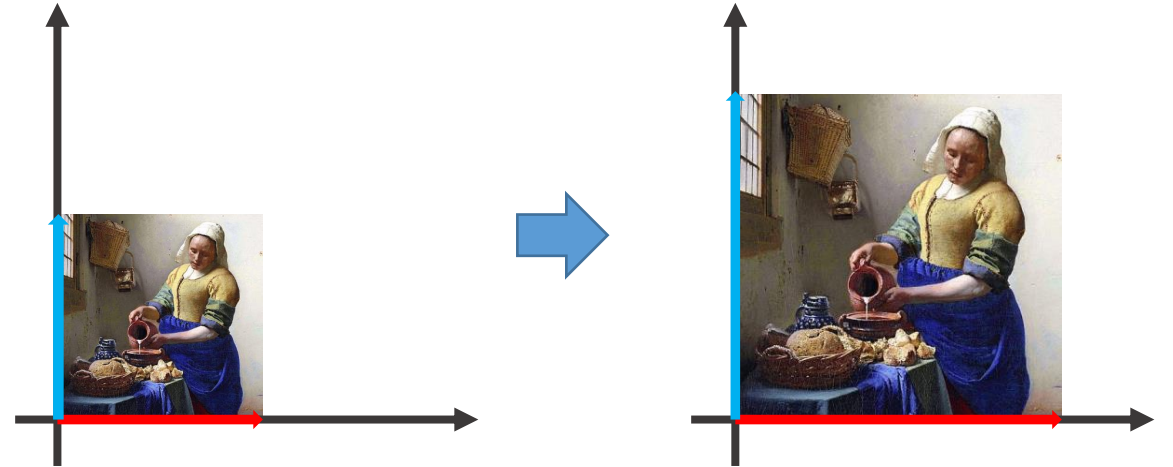
$$\begin{bmatrix} 0.5 & 0 \\ 0 & 2 \end{bmatrix}$$



8-1 線形変換 拡大, 縮小

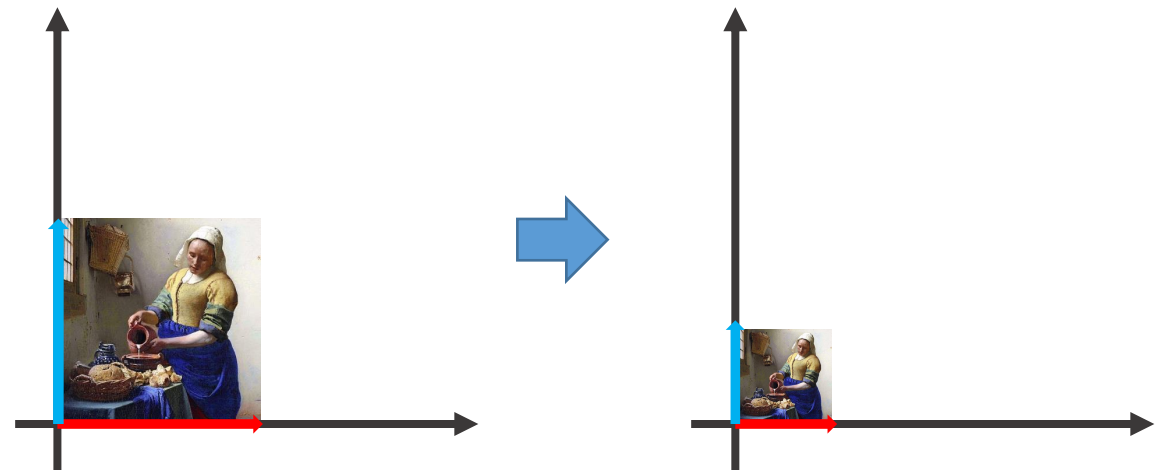
画像の拡大

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



画像の縮小

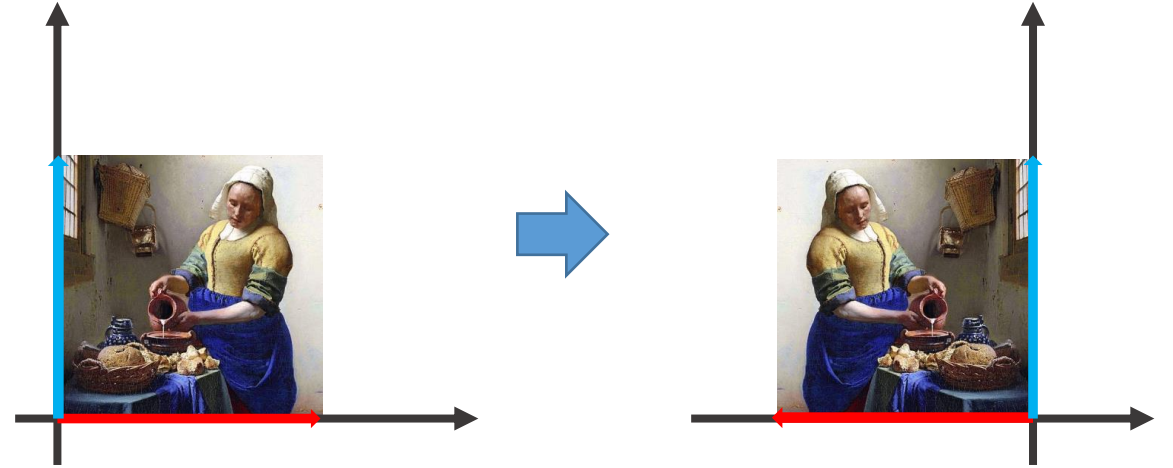
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



8-1 線形変換 鏡映

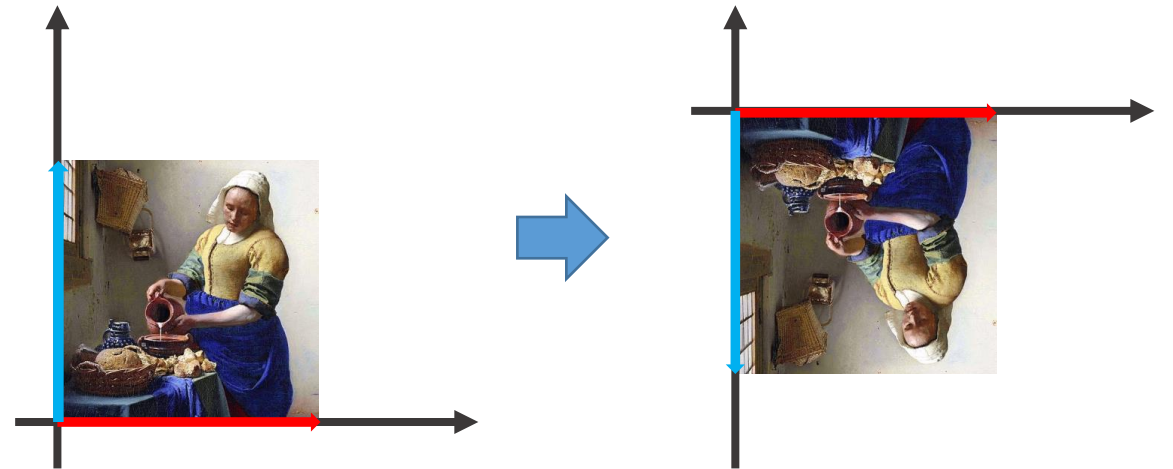
画像の左右反転

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



画像の上下反転

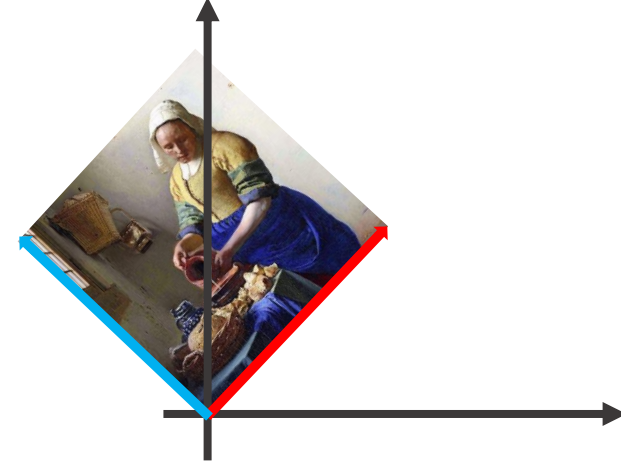
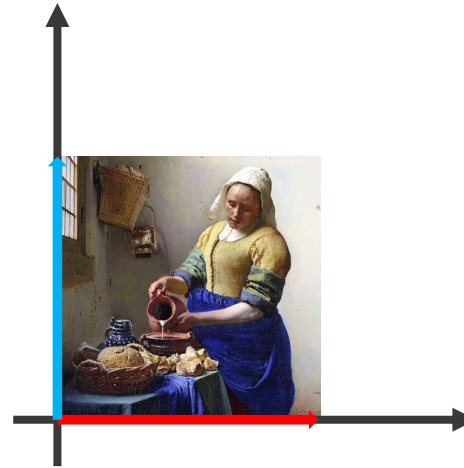
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



8-1 線形変換 回転

画像の回転（画像の左下原点とした場合）

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



→ (cos θ , sin θ)

↑ (cos θ + 90度 , sin θ + 90度)

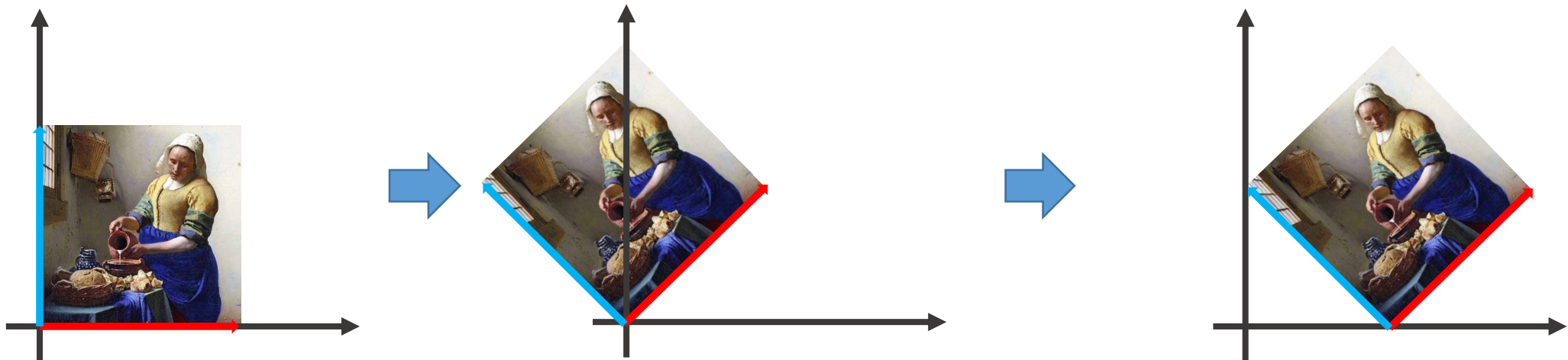
8-2 アフィン変換 平行移動

これら線形変換の行列に平行移動を追加する

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

t_x : x方向への平行移動

t_y : y方向への平行移動



8-2 アフィン変換 平行移動

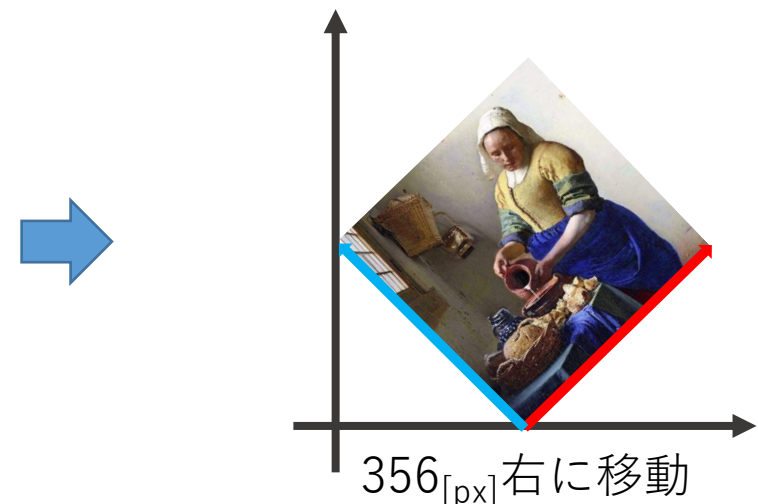
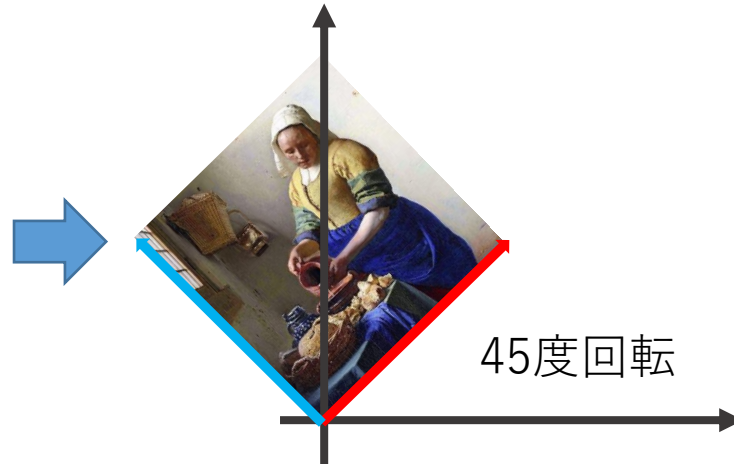
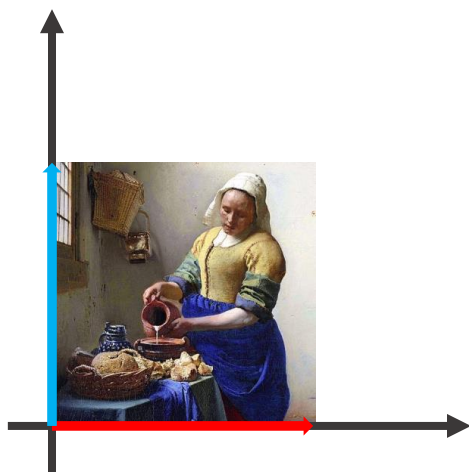
例えば回転 + 平行移動の場合（原点を中心に回転後、第一象限に収める）

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

画像サイズ : $500 \times 500_{\text{px}}$
回転 : 45度

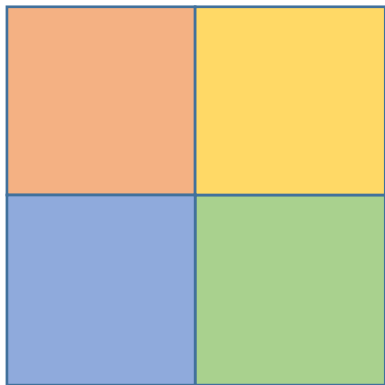
$500 \times \sqrt{2}/2 = 356$ くらい

$\theta = 45$
 $t_x = 356, t_y = 0$



8-3 再標本化 補完

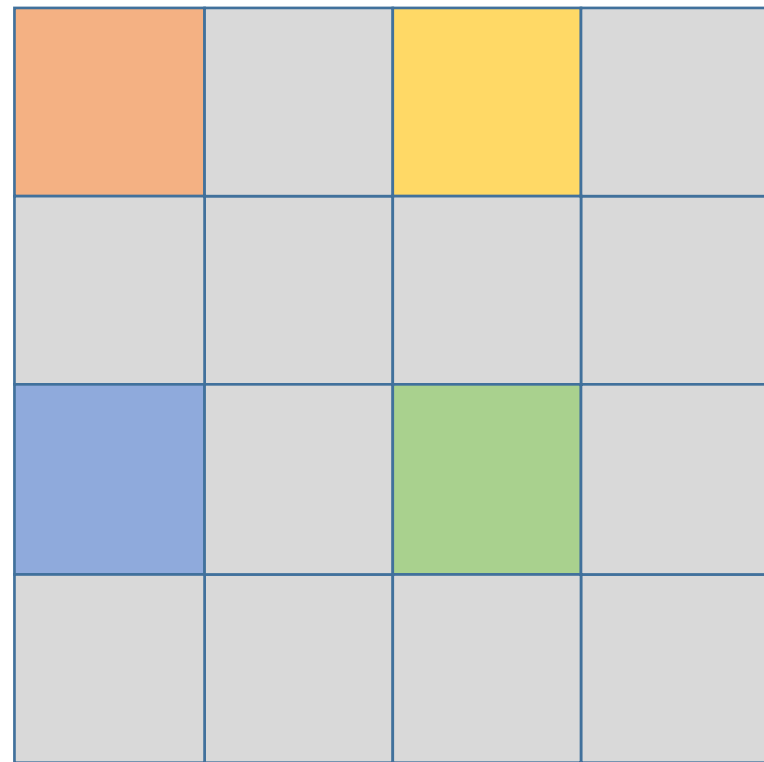
画像を拡大や回転すると穴が生まれる
その穴を埋める必要がある



縦 2倍
横 2倍

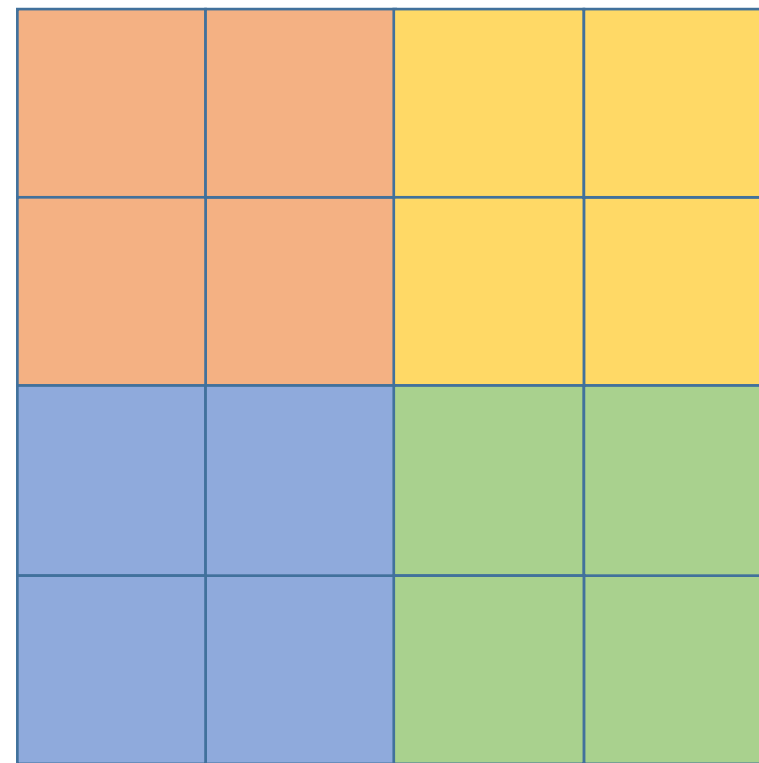
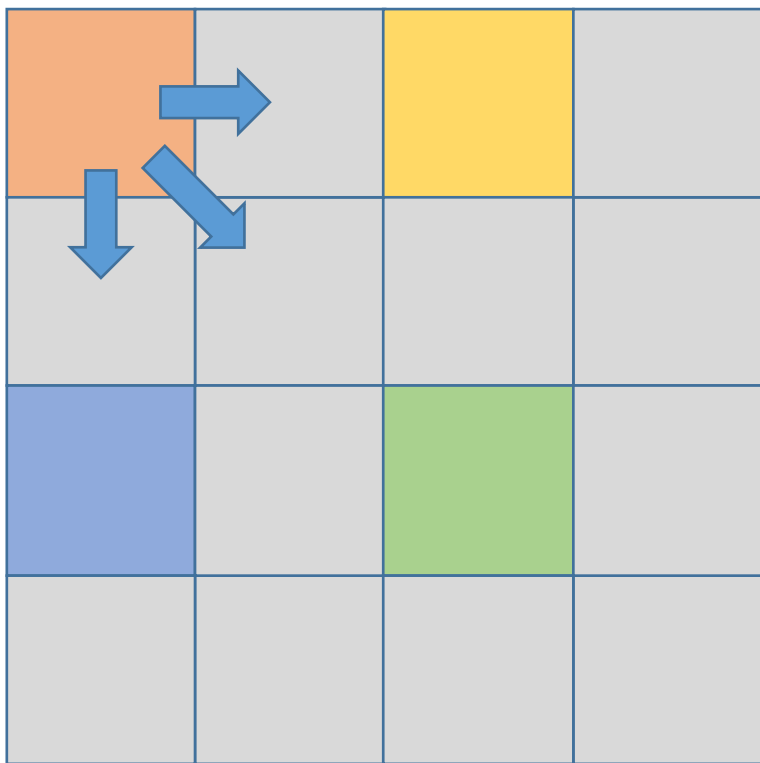


画素のないピクセル



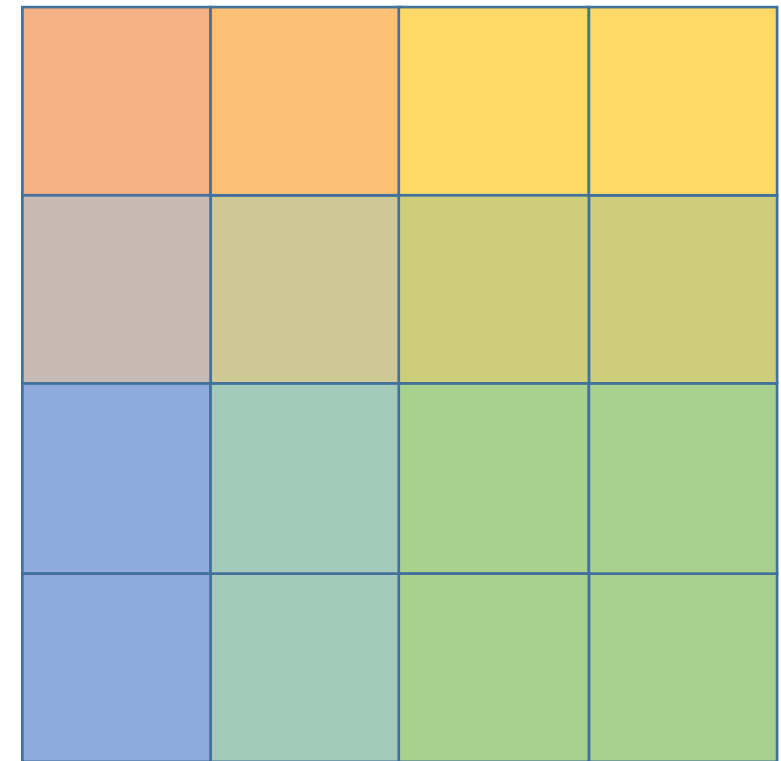
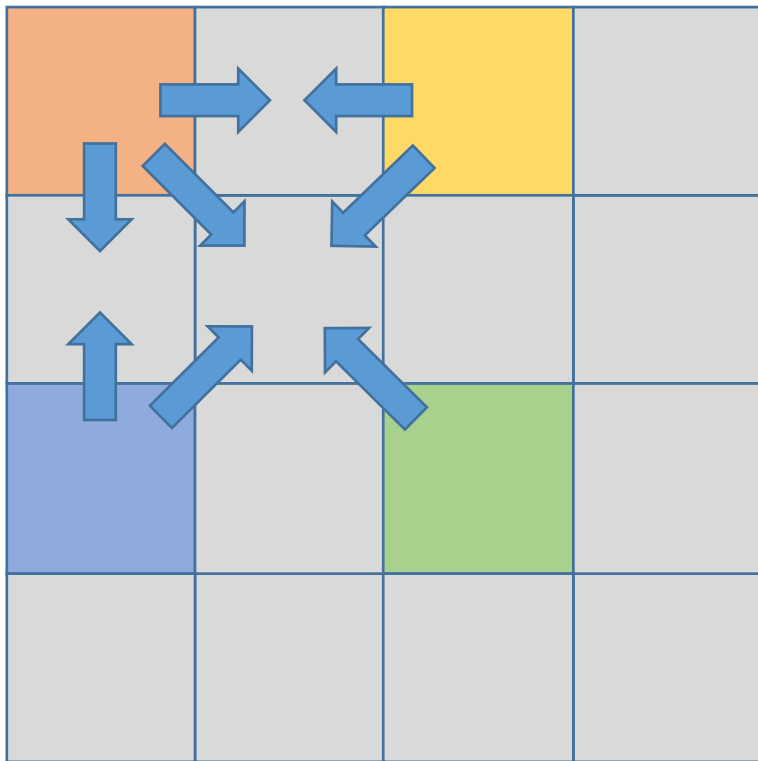
8-3 最標本化 ニアレストネイバー

ニアレストネイバーは一番近い色をそのまま使う補完方法
画像が荒く見える（劣化する）



8-3 最標本化 バイリニア補完

周囲4点からの画素値に距離と反比例する重みをつけて平均化する



8-3 最標本化 バイキュービック補間

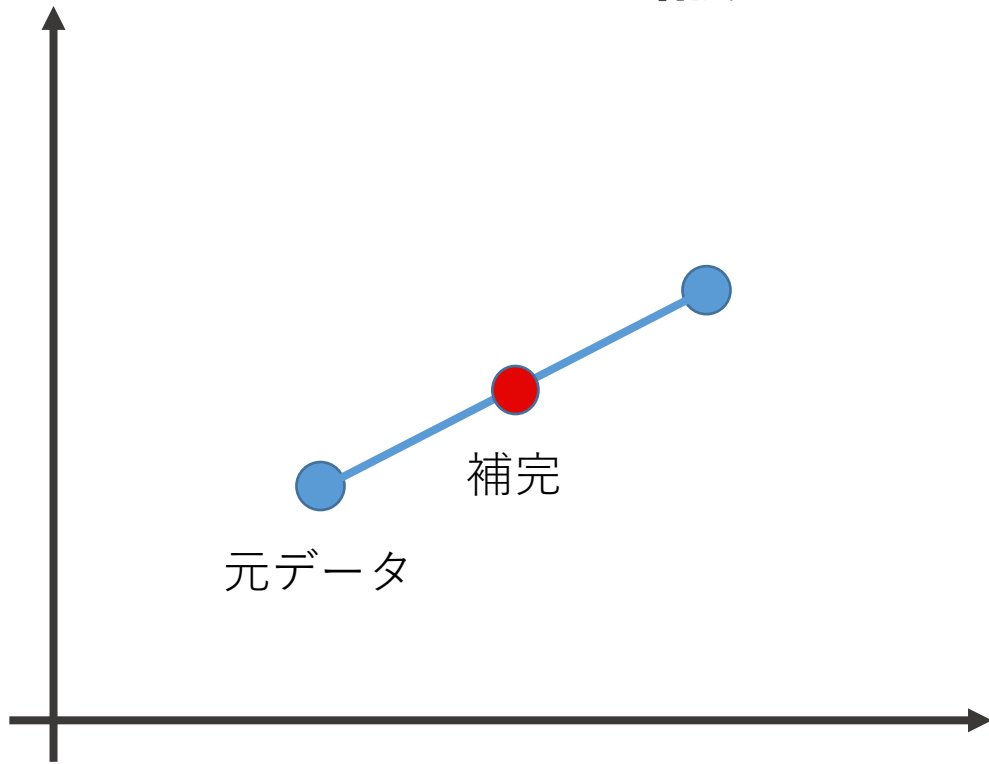
バイリニア補完が周囲 1 マスから計算されるのに対し、
バイキュービック補完は周囲 2 マスから計算される。

参照する画素が増えたので、より滑らかな画像になる。



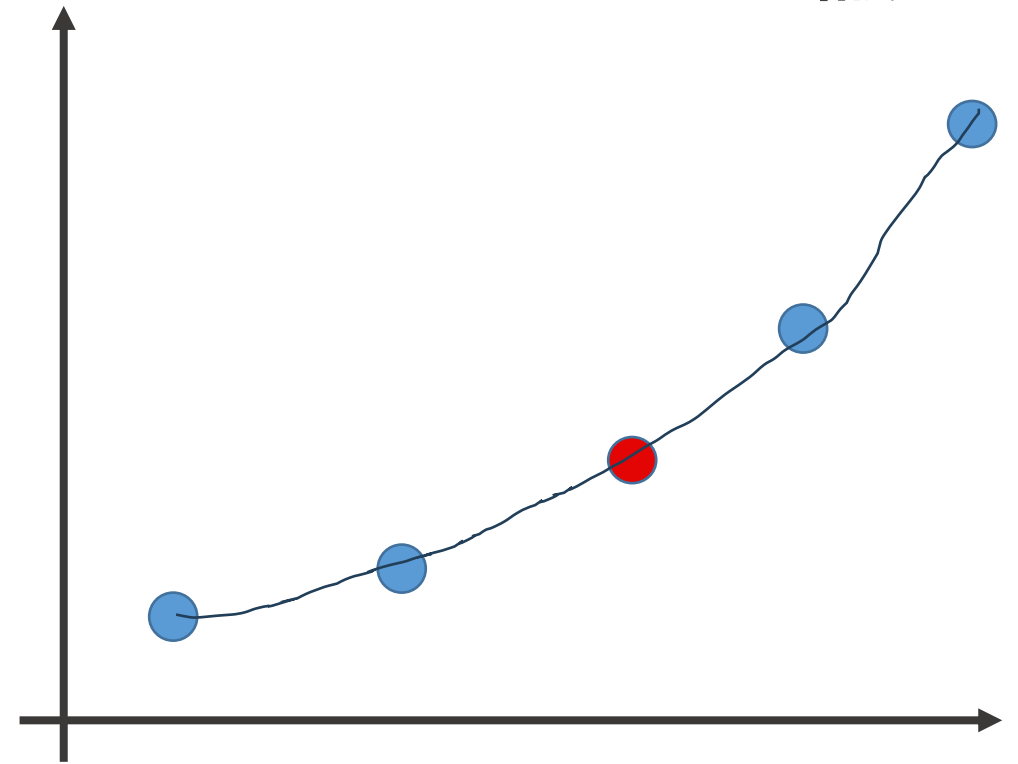
8-3 最標本化 バイリニア補完とバイキュービック補間の違い

バイリニア補完



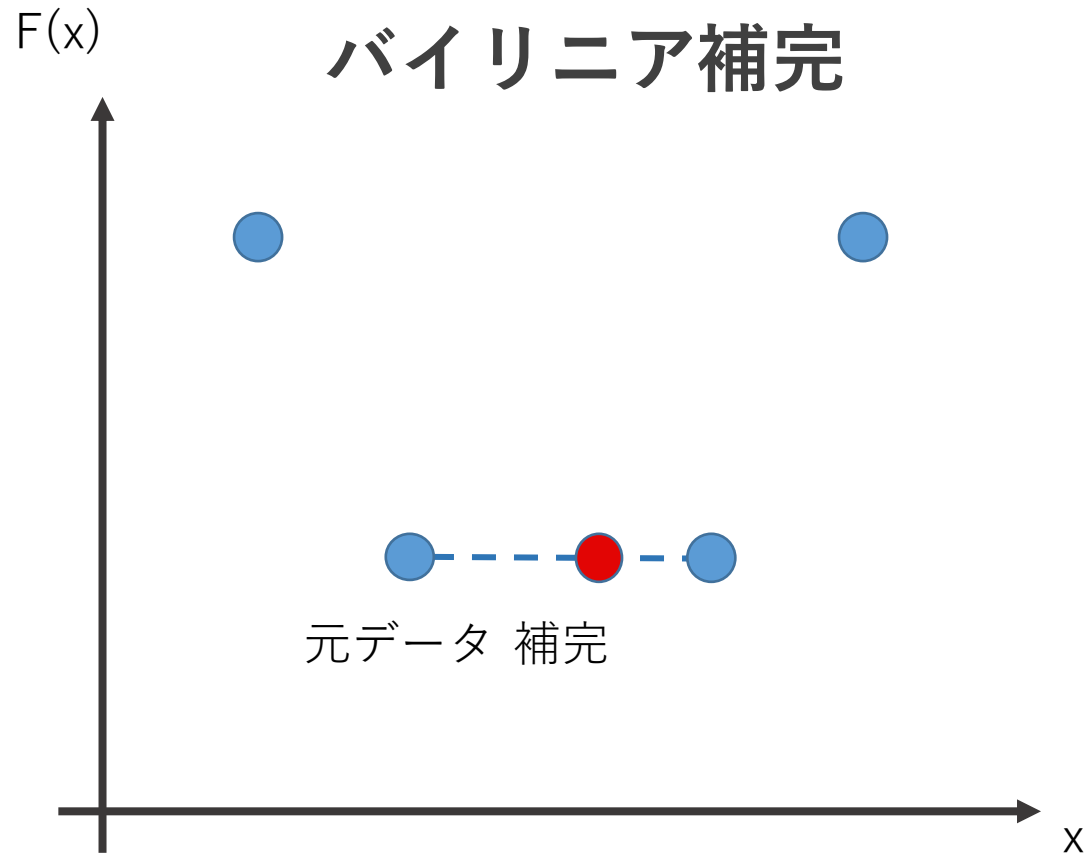
直線で補完（線形補完）

バイキュービック補完

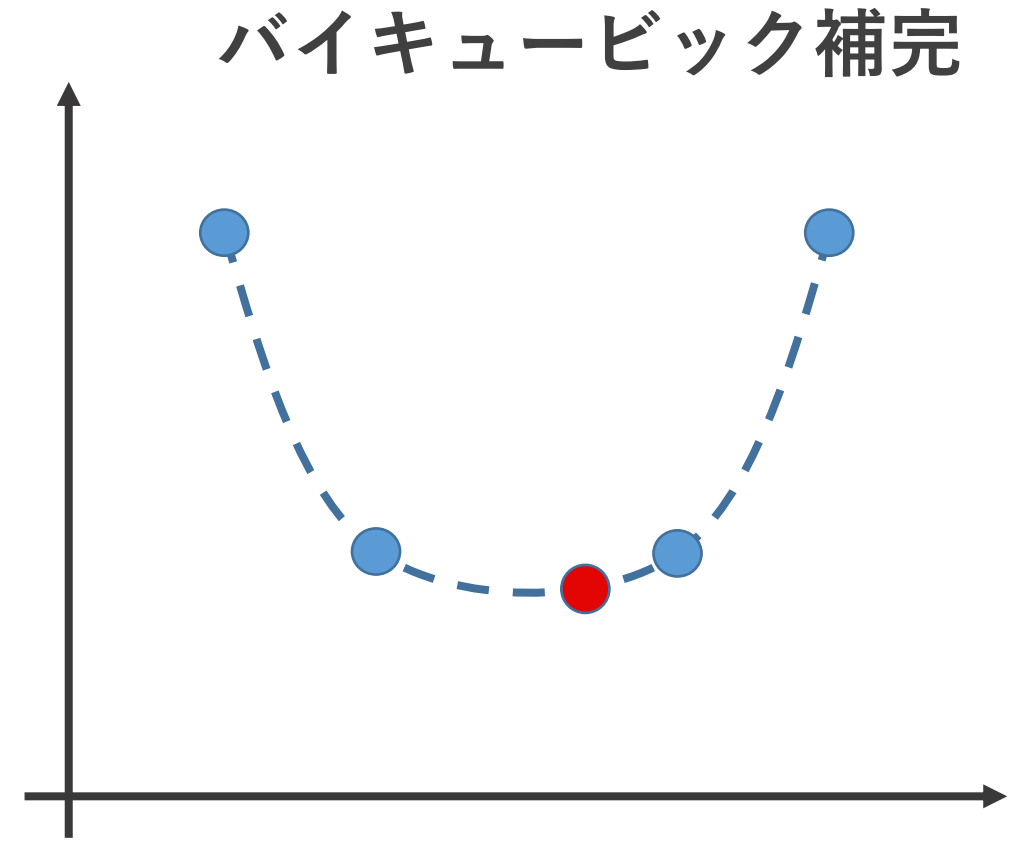


3次多項式で補完

8-3 最標本化 バイリニア補完とバイキュービック補間の違い2



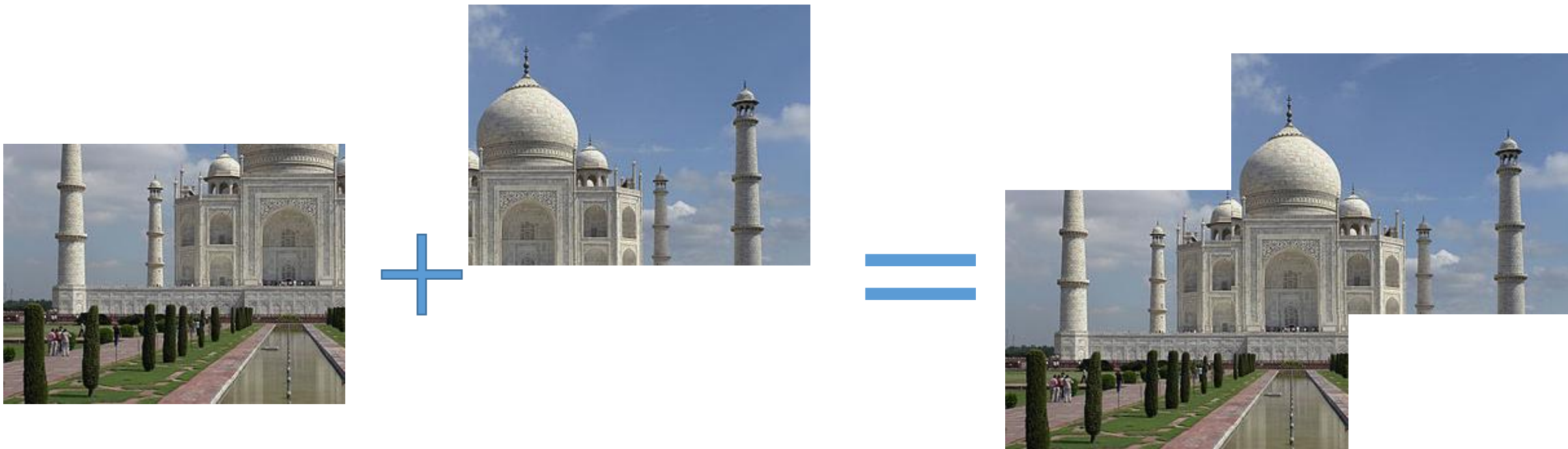
2点で補完



8-4 イメージモザイクキング 概要

画像をつなぎ合わせる技術

パノラマ画像や360度の写真が作成できる



8-4 イメージモザイクング 特徴点

SIFTを利用した特徴点抽出に関する詳細情報に関しては以下を参照

教科書11-4 特徴点の記述とマッチング

論文「Gradient ベースの特徴抽出 -SIFT と HOG」
図2, 8, 9, 10, 11, 12を見ると特に理解しやすい。

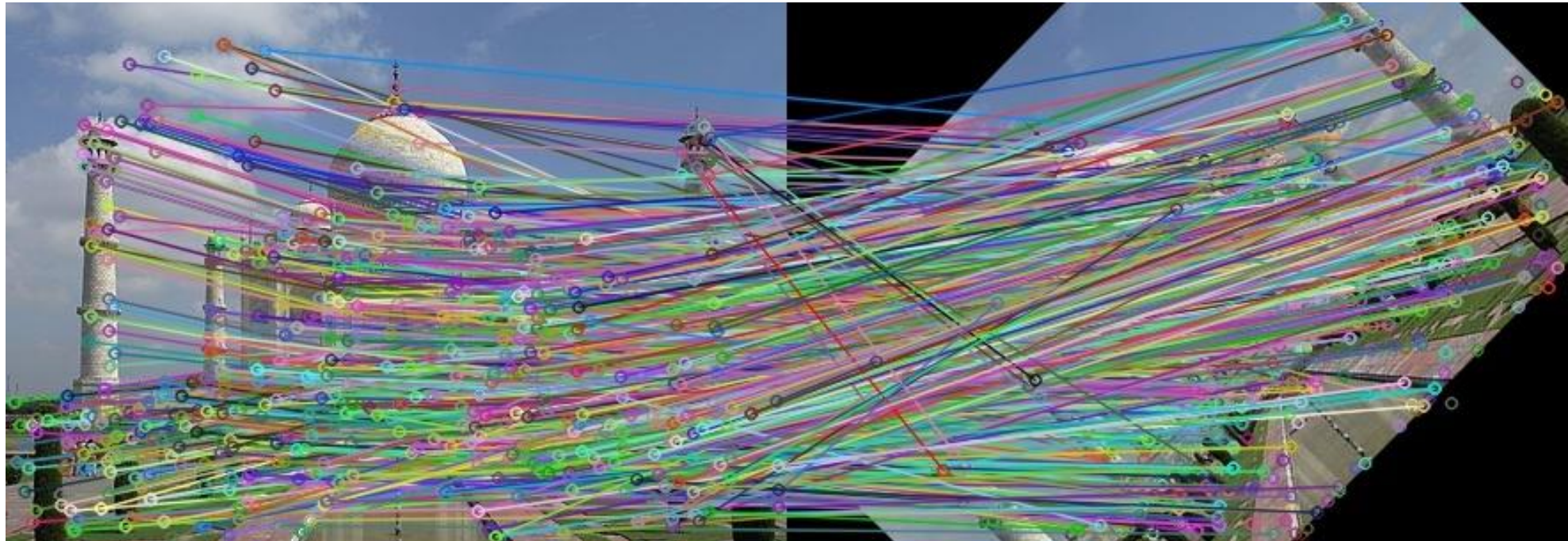
特徴点を求めるアルゴリズムは他にもいくつかある。

(例) AKAZE 特徴量, BRISK 特徴量, KAZE 特徴量, ORB 特徴量

8-4 イメージモザイクシング SIFT

SIFTを利用してマッチングを行う。

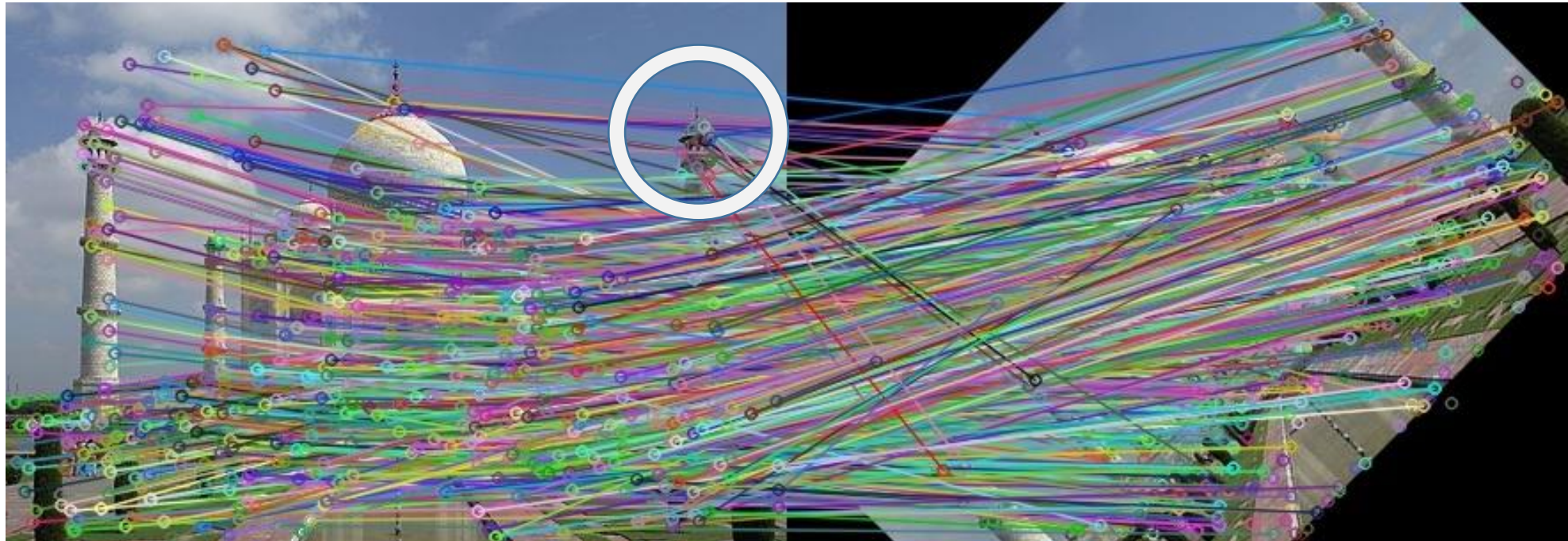
SIFTの特徴として、
明るさの違い、スケール・回転変化の影響を受けづらい



8-4 イメージモザイクング アウトライヤ

以下の画像には間違ったマッチングがある。
これをアウトライヤという。

例えばここ



演習 特徴点 07-1

```
//特徴点検出
//検出方法の指定とモジュール生成
cv::Ptr<cv::Feature2D> features = cv::SIFT::create();

//画像毎の特徴点を格納するための配列
std::vector<cv::KeyPoint> keyPoints[2];
//画像毎の特徴量を格納するための配列
cv::Mat descriptors[2];

// それぞれの入力画像に対してdetectAndCompute関数を用いて特徴点と特徴量を算出
features->detectAndCompute(sourceGray, cv::noArray(), keyPoints[0], descriptors[0]);
features->detectAndCompute(rotationGray, cv::noArray(), keyPoints[1], descriptors[1]);

//特徴量マッチング
cv::BFMatcher matcher;

//最も良い点を探す
std::vector<cv::DMatch> matches;
matcher.match(descriptors[0], descriptors[1], matches);

//見つかった対応点の表示
cv::drawMatches(sourceImage, keyPoints[0], rotationImg, keyPoints[1], matches, resultImage);
```


8-4 イメージモザイク RANSAC

アウトライヤを消すためにRANSACと呼ばれる処理をする

1. 対応点をランダムに数個選ぶ.
2. その点から射影変換のパラメータを作成
3. そのパラメータを使用し他の対応点が
作成したパラメータ通りに対応しているか確認

*a. 例えば2で「画像は45度回転している」パラメータができる
3では他の点も45度回転しているかチェックする

*b. このとき正しく対応している点をインライアという

4. 3で対応している対応点が最も多いパターンを採用

8-4 イメージモザイク インライア

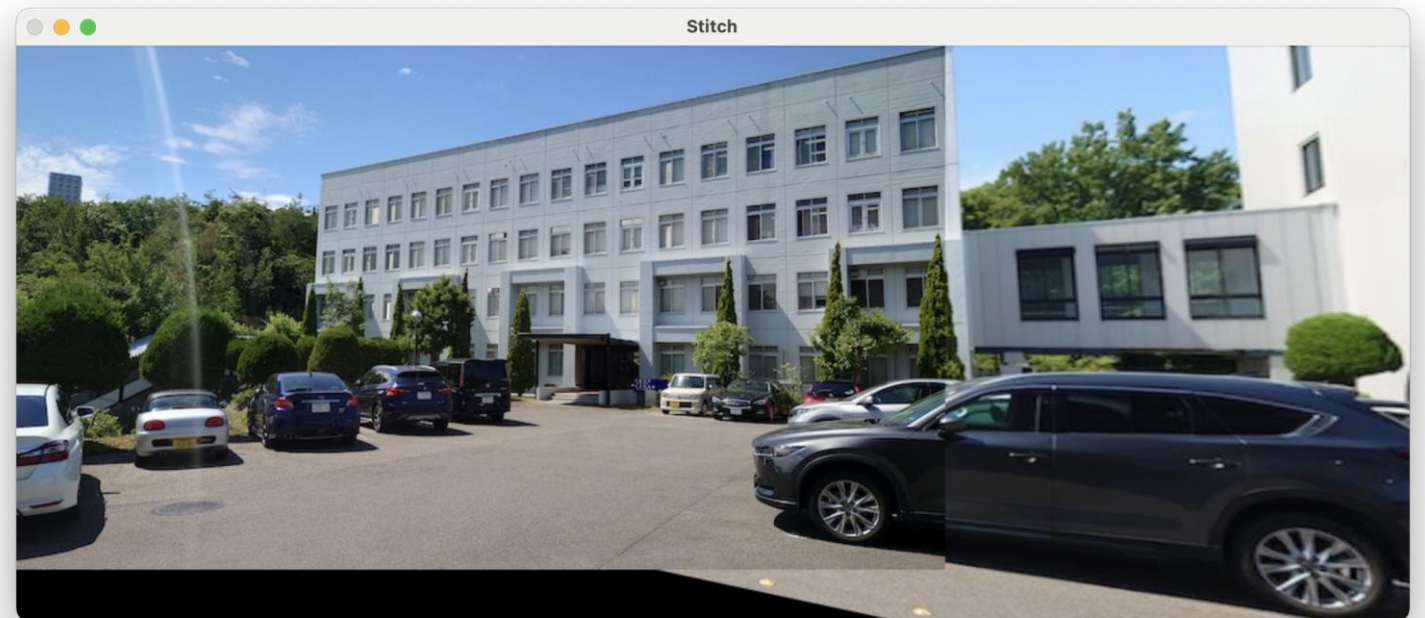
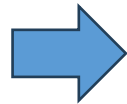
アウトライアを消すためにRANSACし、
最もインライアとして対応点が多かったパターンが出力される。



8-4 イメージモザイクキングパノラマ画像

対応点を利用してパノラマ画像を作成
以下の画像は2枚で作成した場合のパノラマ画像です。

入力画像は、panorama1.jpgとpanorama2.jpgです。



演習 対応点選別 07-2 (Siftバージョン)

```
//上位2個の良い点を探す
std::vector<std::vector<cv::DMatch>> knnMatches;
matcher.knnMatch(descriptors[0], descriptors[1], knnMatches, 2);

//対応点を絞る
const double match_par = 0.6; //候補点を残す場合のしきい値
std::vector<cv::DMatch> goodMatch;

//KeyPoint -> Point2f
std::vector<cv::Point2f> match_point[2];
for (size_t i = 0; i < knnMatches.size(); ++i) {
    double distance1 = knnMatches[i][0].distance;
    double distance2 = knnMatches[i][1].distance;

    //第二候補点から距離が離れている点のみ抽出 (いい点だけ残す)
    if (distance1 <= distance2 * match_par) {
        goodMatch.push_back(knnMatches[i][0]);
        match_point[0].push_back(keyPoints[0][knnMatches[i][0].queryIdx].pt);
        match_point[1].push_back(keyPoints[1][knnMatches[i][0].trainIdx].pt);
    }
}
```

演習 対応点選別 07-2 (Siftバージョン)

```
//ホモグラフィ行列推定(RANSAC)
cv::Mat masks;
cv::Mat H = cv::findHomography(match_point[0], match_point[1], masks, cv::RANSAC, 3);

//RANSACで使われた対応点のみ抽出
std::vector<cv::DMatch> inlinerMatch;
for (size_t i = 0; i < masks.rows; ++i) {
    uchar *inliner = masks.ptr<uchar>(i);
    if (inliner[0] == 1) {
        inlinerMatch.push_back(goodMatch[i]);
    }
}

cv::Mat inlinerImage;

//対応点を絞った結果を表示
cv::drawMatches(Sources[0], keyPoints[0], Sources[1], keyPoints[1], goodMatch, resultImage);
//インライアの対応点のみ表示
cv::drawMatches(Sources[0], keyPoints[0], Sources[1], keyPoints[1], inlinerMatch, inlinerImage);
```

演習 ステッチング 07-3 (Siftバージョン)

//画像を変形して繋ぎ合わせる

```
cv::warpPerspective(Sources[0], resultImage, Homo, cv::Size(static_cast<int>(Sources[0].cols * 1.5),
    static_cast<int>(Sources[0].rows * 1.1)));
for (int y = 0; y < Sources[0].rows; y++){
    for (int x = 0; x < Sources[0].cols; x++){
        resultImage.at<cv::Vec3b>(y, x) = Sources[1].at<cv::Vec3b>(y, x);
    }
}
```

演習 スティッチング 07-3 (Akazeバージョン)

Siftでは精度が良くないことがある。
より頑健にしたものとして、Akazeがあります。

入力画像は、panorama1.jpgとpanorama2.jpgです。

ソースはmc07-akaze.cpp

実行して確認しましょう。

課題 三枚以上の画像のステッチング

- 二枚のステッチングをやりましたので，三枚（以上）の画像ステッチングをしてください.
- 画像はimagesのものを使っても良いし，自分で撮影したものでも良い.
- 元画像と結果画像をzip圧縮して提出してください.